WIE

Flux Media Service

Complete API Documentation for React Native Developers

Base URL: http://localhost:5010/api • All routes require Bearer token • v1.0

Table of Contents

01 Service Overview & Base URL

The WIE Media Service runs on port 5010 and manages all Flux (stories), Diary (highlights), Music, and Location features. Every request must carry a valid Bearer token obtained during login.

Service Config

```
Base URL (development) : http://localhost:5010/api
Base URL (Android emu) : http://10.0.2.2:5010/api

Route prefixes:
  Flux       →  /api/flux
  Diary      →  /api/diary
  Music      →  /api/music
  Location   →  /api/location
```

▸ Flux Lifecycle

A Flux (story) lives for 24 hours by default (configurable to 48 or 72 h via settings). After expiry it becomes archived automatically if the user has archive-save enabled, or is soft-deleted if not. Persistent stories are pinned with no expiry (up to 5 per user).

Flux Data Model

```
Flux Status values:
  active    →  currently live and visible
  expired   →  past expiry, moved to archive
  deleted   →  soft-deleted (hidden from all queries)

Flux Visibility values:
  public        →  anyone can view
  followers     →  only followers
  close_friends →  only users in close-friends list
  only_me       →  only the owner
```

02 Authentication

Every endpoint in this service requires authentication. The middleware reads the token from the Authorization header and attaches req.userId to the request.

TypeScript — Axios Instance

```
// Attach token to every request (React Native)
import AsyncStorage from '@react-native-async-storage/async-storage';
import axios from 'axios';

const mediaApi = axios.create({
  baseURL: 'http://10.0.2.2:5010/api',
  timeout: 20000,
```

```
});

mediaApi.interceptors.request.use(async (config) => {
  const token = await AsyncStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

> All 401 responses mean the token is expired or missing. Clear AsyncStorage and redirect to Login.

03 Flux Creation

Creating a flux is the central action of the stories feature. The endpoint accepts multipart/form-data so media files (images or videos) can be uploaded alongside metadata.

Method	Endpoint	Auth	Description
POST	/api/flux/create	✅ Yes	Create a new flux (story). Accepts media file or text-only.

▸ Request — multipart/form-data fields

Field	Type	Required	Description
media	File	No	Image or video file (jpg/png/gif/webp/mp4/mov). Omit for text-only story.
caption	string	No	Caption text shown below media.
visibility	string	No	public | followers | close_friends | only_me. Default: public
textLayers	JSON[]	No	Array of text layer objects for text stories (stringified JSON).
textBg	string	No	Background colour for text story (hex or gradient string).
filterName	string	No	CSS filter name (e.g. 'Clarendon'). Default: Normal
filterValue	string	No	CSS filter value string. Default: none
musicId	string	No	iTunes / Spotify track ID.
musicTitle	string	No	Track title for display.
musicArtist	string	No	Artist name for display.
musicStartAt	number	No	Playback start offset in seconds.
musicPreviewUrl	string	No	30-second audio preview URL.
musicAlbumArt	string	No	Album artwork URL.
locationLabel	string	No	Display label for location sticker.
locationPlaceId	string	No	Google Place ID.
locationLat	number	No	Latitude (decimal degrees).
locationLng	number	No	Longitude (decimal degrees).
locationCategory	string	No	Place category (e.g. 'restaurant').
locationStickerX	number	No	Sticker X position as % (0–100). Default: 50
locationStickerY	number	No	Sticker Y position as % (0–100). Default: 75
stickers	JSON[]	No	Array of sticker overlay objects (stringified JSON).
overlays	JSON[]	No	Array of custom overlay objects (stringified JSON).
mentions	string[]	No	Array of user IDs to mention (stringified JSON).
duration	number	No	Story duration override: 15 | 30 | 60 (seconds for video trim).

▸ Response

JSON Response

```
{
  "success": true,
  "data": {
    "_id": "flux_abc123",
    "userId": "user_xyz",
    "mediaUrl": "https://res.cloudinary.com/.../story.mp4",
    "mediaType": "video",
    "caption": "At the beach!",
    "visibility": "public",
    "status": "active",
    "expiresAt": "2025-12-02T10:00:00.000Z",
    "createdAt": "2025-12-01T10:00:00.000Z",
    "viewCount": 0,
    "reactionCount": 0
  }
}
```

▸ React Native — Image / Video Upload

TypeScript

```
import { launchImageLibrary } from 'react-native-image-picker';

const createFlux = async () => {
  const result = await launchImageLibrary({ mediaType: 'mixed', quality: 0.85 });
  if (!result.assets?.[0]) return;
  const asset = result.assets[0];

  const formData = new FormData();
  // ☑ React Native FormData — use uri/name/type, NOT File/Blob
  formData.append('media', {
    uri:  asset.uri,
    name: asset.fileName || 'flux.jpg',
    type: asset.type   || 'image/jpeg',
  } as any);
  formData.append('caption',    'At the beach!');
  formData.append('visibility', 'public');

  // Optional music
  formData.append('musicTitle', 'Shape of You');
  formData.append('musicArtist', 'Ed Sheeran');
  formData.append('musicStartAt', '12');

  const res = await mediaApi.post('/flux/create', formData, {
    headers: { 'Content-Type': 'multipart/form-data' },
    onUploadProgress: (e) => {
      const pct = Math.round((e.loaded * 100) / (e.total || 1));
      setUploadProgress(pct); // show progress bar
    },
  });
  return res.data.data; // Flux object
};
```

▸ Text-Only Flux (no media file)

TypeScript

```
// Text story — omit the 'media' field entirely
const formData = new FormData();
formData.append('textBg', '#8860D9');
formData.append('textLayers', JSON.stringify([
  { text: 'Hello World', x: 50, y: 50, fontSize: 32,
    color: '#FFFFFF', fontFamily: 'Bold' }
]));
formData.append('visibility', 'followers');

const res = await mediaApi.post('/flux/create', formData, {
  headers: { 'Content-Type': 'multipart/form-data' },
});
```

04 Flux Feed — Home Screen Stories

The feed returns all active fluxes from users the viewer follows, plus the viewer's own fluxes. Results are grouped by owner, ordered by most recent. Close-friends fluxes are automatically filtered — only users in the owner's close-friends list see them.

Method	Endpoint	Auth	Description
GET	/api/flux/feed	✅ Yes	Returns grouped feed of active stories for the home screen.
POST	/api/flux/invalidate-feed	✅ Yes	Force-bust the Redis feed cache after a new follow event.

▸ GET /api/flux/feed — Response

JSON Response

```
{
  "success": true,
  "data": [
    {
      "_id": "user_abc",           // owner user ID
      "fluxes": [                    // array of their active stories
        {
          "_id": "flux_001",
          "mediaUrl": "https://cdn.../story.jpg",
          "mediaType": "image",
          "caption": "Sunset vibes",
          "visibility": "public",
          "viewCount": 34,
          "createdAt": "2025-12-01T08:00:00.000Z",
          "expiresAt": "2025-12-02T08:00:00.000Z"
        }
      ],
      "latestAt": "2025-12-01T08:00:00.000Z",
      "isSelf": false,              // true if this is the viewer's own group
      "user": {
        "id": "user_abc",
        "username": "janedoe",
        "name": "Jane Doe",
        "profile_picture": "https://cdn.../avatar.jpg",
        "is_verified": false
      }
    }
  ]
}
```

▸ POST /api/flux/invalidate-feed — Body

JSON Body

```
// Call this immediately after the user follows someone new
// so their fluxes appear in the feed without waiting for cache TTL
```

```
{ "ownerId": "<id of the newly followed user>" }
```

> The feed is Redis-cached for 60 seconds. The isSelf=true group should always appear first in your UI — treat it as the 'Your Story' bubble.

05 My Fluxes, Single Flux & User Fluxes

Method	Endpoint	Auth	Description
GET	/api/flux/mine	✅ Yes	Own ACTIVE fluxes only (avatar story ring). Sorted oldest-first.
GET	/api/flux/all-mine	✅ Yes	All own fluxes incl. expired & archived (for diary/highlights picker).
GET	/api/flux/archive	✅ Yes	Archived (expired) + persistent (pinned) stories.
GET	/api/flux/user/:userId	✅ Yes	Active fluxes for any other user (respects visibility rules).
GET	/api/flux/:fluxId	✅ Yes	Single flux by ID (enforces close_friends / only_me access).
DELETE	/api/flux/:fluxId	✅ Yes	Soft-delete a flux (owner only). Media is removed from Cloudinary.

▸ GET /api/flux/mine — Response

JSON Response

```
{
  "success": true,
  "data": [ /* Flux[] — only active, non-expired */ ],
  "count": 3
}
```

▸ GET /api/flux/archive — Response

JSON Response

```
{
  "success": true,
  "data": {
    "expired": [ /* Flux[] — isArchived=true, isPersistent=false */ ],
    "active":  [ /* Flux[] — isPersistent=true, never expires  */ ]
  }
}
```

▸ Story Ring Logic (Profile Avatar)

TypeScript

```
// Determine whether to show gradient ring on profile avatar
const myFluxes = await mediaApi.get('/flux/mine');
const hasActiveStory = myFluxes.data.data.length > 0;

// In React Native — LinearGradient ring wrapper
import LinearGradient from 'react-native-linear-gradient';
```

```
const AvatarWithRing = ({ uri, fluxes, onPress }) => (
  hasActiveStory ? (
    <TouchableOpacity onPress={onPress}>
      <LinearGradient
        colors={['#8860D9','#B3B8E2','#2979FF']}
        style={{ padding: 3, borderRadius: 999 }}
      >
        <Image source={{ uri }} style={styles.avatar} />
      </LinearGradient>
    </TouchableOpacity>
  ) : <Image source={{ uri }} style={styles.avatar} />
);
```

06 Viewing Fluxes

When a user watches a story two endpoints are relevant: record-view (deduped count) and viewers (owner sees who watched). Always call record-view when displaying a story.

Method	Endpoint	Auth	Description
POST	/api/flux/:fluxId/view	✅ Yes	Legacy view — adds to views subdoc array (analytics).
POST	/api/flux/:fluxId/record-view	✅ Yes	Deduped view via $addToSet. Use this in the flux player.
GET	/api/flux/:fluxId/viewers	✅ Yes	Full viewer list + reactions (owner only).
GET	/api/flux/:fluxId/view-list	✅ Yes	Raw viewer ID list + total count (owner only).

▸ POST /api/flux/:fluxId/record-view — Response

JSON Response

```
{ "success": true, "viewCount": 42 }
```

▸ GET /api/flux/:fluxId/viewers — Response (owner)

JSON Response

```
{
  "success": true,
  "total": 40,
  "viewCount": 40,
  "viewers": [
    {
      "id": "user_abc",
      "username": "janedoe",
      "name": "Jane Doe",
      "profile_picture": "https://cdn.../avatar.jpg",
      "is_verified": false
    }
  ],
  "reactions": [
    { "userId": "user_abc", "emoji": "😍", "createdAt": "..." }
  ]
}
```

▸ Flux Player — Call Pattern

TypeScript

```
// Call record-view as soon as the story becomes visible
// (e.g. in onViewableItemsChanged or useEffect when index changes)
const onStoryVisible = async (fluxId: string) => {
```

```
  try {
    const res = await mediaApi.post(`/flux/${fluxId}/record-view`);
    setViewCount(res.data.viewCount);
  } catch { /* silent — never block playback on view failure */ }
};
```

07 Reactions & Likes

WIE supports two ways to express sentiment on a flux: emoji reactions (shown in the viewer list for the owner) and likes (heart tap). Reactions and likes are disabled if the owner's settings forbid them.

Method	Endpoint	Auth	Description
POST	/api/flux/:fluxId/react	✅ Yes	React with an emoji. Only non-owners. Replaces previous reaction.
POST	/api/flux/:fluxId/like	✅ Yes	Toggle like/unlike. Sends notification to owner. Non-owners only.
GET	/api/flux/:fluxId/likes	✅ Yes	Get likes. Owner gets full list; non-owner gets count + hasLiked.

▸ POST /api/flux/:fluxId/react — Body

JSON Body

```
{ "emoji": "😍" }   // any Unicode emoji string
```

▸ POST /api/flux/:fluxId/like — Body & Response

JSON

```
// Body (emoji is optional, defaults to ♡)
{ "emoji": "♡" }

// Response
{
  "success": true,
  "liked": true,       // false when unlike
  "likeCount": 15
}
```

▸ GET /api/flux/:fluxId/likes — Response

JSON Response

```
// Non-owner response
{ "success": true, "total": 15, "hasLiked": true }

// Owner response
{
  "success": true,
  "total": 15,
  "likes": [
    { "userId": "u1", "emoji": "♡", "name": "Jane",
      "avatar": "https://...", "createdAt": "..." }
  ]
}
```

> Reactions appear in the owner's viewers panel. Likes are separate. A user can both react AND like the same flux.

08 Flux Comments

Comments can be enabled/disabled per flux by the owner. The allowReplies setting controls who can comment: 'everyone', 'mutual' (both must follow each other), or 'off'.

Method	Endpoint	Auth	Description
POST	/api/flux/:fluxId/comments	✅ Yes	Add a comment to a flux.
GET	/api/flux/:fluxId/comments	✅ Yes	Get all comments (enriched with user info).
POST	/api/flux/:fluxId/comments/:commentId/like	✅ Yes	Toggle like on a comment.
PATCH	/api/flux/:fluxId/comments/toggle	✅ Yes	Owner toggles comments on/off for this flux.

▸ POST /api/flux/:fluxId/comments — Body

JSON Body

```
{ "text": "Amazing shot! 🔥" }
```

▸ GET /api/flux/:fluxId/comments — Response

JSON Response

```
{
  "success": true,
  "total": 4,
  "comments": [
    {
      "_id": "comment_001",
      "userId": "user_abc",
      "text": "Amazing shot! 🔥",
      "likes": ["user_xyz"],
      "createdAt": "2025-12-01T11:00:00.000Z",
      "name": "Jane Doe",
      "username": "janedoe",
      "avatar": "https://cdn.../avatar.jpg"
    }
  ]
}
```

09 Flux Replies (Chat-Based)

A flux reply is sent as a special chat message to the flux owner's DM thread — not stored on the flux document. The chat service handles delivery and notifications automatically.

Method	Endpoint	Auth	Description
POST	/api/flux/:fluxId/reply	✅ Yes	Send a text reply to a flux (delivered as a DM to the owner).

▸ POST /api/flux/:fluxId/reply — Body

JSON Body

```
{ "text": "Where is this place? Looks amazing!" }
```

▸ Response

JSON Response

```
{
  "success": true,
  "message": {
    "chat_id": "chat_abc",
    "message_id": "msg_xyz"
  }
}
```

> Replies are blocked if allowReplies='off' or allowMessageReplies=false in the flux owner's settings. Check getFluxOwnerSettings first.

10 Flux Sharing

A flux can be shared to multiple recipients as a DM at once. Each recipient gets a chat message containing the flux preview. Sharing is blocked if the owner has disabled it.

Method	Endpoint	Auth	Description
POST	/api/flux/:fluxId/share	✅ Yes	Share flux to one or more user IDs as chat messages.

▸ POST /api/flux/:fluxId/share — Body

JSON Body

```
{
  "receiverIds": ["user_001", "user_002", "user_003"]
}
```

▸ Response

JSON Response

```
{
  "success": true,
  "message": "Flux shared",
  "results": [
    { "receiverId": "user_001", "chatId": "chat_a", "messageId": "msg_1" },
    { "receiverId": "user_002", "error": "Failed to send" }
  ]
}
```

11 Mentions & Re-Mentions

The flux owner can mention other users in their story. Mentioned users see the flux in their story feed. They can re-mention it (reshare to their own followers). All mention events generate notifications and DM chat messages automatically.

Method	Endpoint	Auth	Description
POST	/api/flux/:fluxId/mention	✅ Yes	Mention users in this flux. Owner only. Must follow mentioned users.
GET	/api/flux/:fluxId/mentions	✅ Yes	Get all mentioned users with their profile info.
DELETE	/api/flux/:fluxId/mention/remove	✅ Yes	Mentioned user removes themselves from this flux (soft).
POST	/api/flux/:fluxId/remention	✅ Yes	Mentioned user reshares the flux (re-mention).
GET	/api/flux/:fluxId/rementions	✅ Yes	Get list of users who have re-mentioned this flux.
GET	/api/flux/:fluxId/permissions	✅ Yes	Check isOwner + isMentioned for current viewer.

▸ POST /api/flux/:fluxId/mention — Body

JSON Body

```
{
  "mentionedUserIds": ["user_abc", "user_def"]
}

// Rules enforced server-side:
// 1. You must follow the mentioned user
// 2. You cannot mention yourself
// 3. If target has a private account, they must follow you back (mutual)
// 4. Each user can only be mentioned once per flux
```

▸ GET /api/flux/:fluxId/mentions — Response

JSON Response

```
{
  "success": true,
  "total": 2,
  "mentions": [
    {
      "userId": "user_abc",
      "name": "Jane Doe",
      "username": "janedoe",
      "avatar": "https://cdn.../avatar.jpg",
      "createdAt": "..."
```

```
    }
  ]
}
```

▸ POST /api/flux/:fluxId/remention — Body

JSON Body

```
{ "comment": "Check this out! 🔥" }  // optional comment
```

▸ GET /api/flux/:fluxId/permissions — Response

JSON Response

```
{
  "success": true,
  "isOwner": false,
  "isMentioned": true,
  "ownerId": "user_xyz"
}
```

12 Flux Settings

Each user has a single FluxSettings document that controls default behaviour for all their stories. Settings are applied server-side when creating or interacting with fluxes.

Method	Endpoint	Auth	Description
GET	/api/flux/settings	✅ Yes	Get current user's story settings.
PATCH	/api/flux/settings	✅ Yes	Update one or more settings fields.
GET	/api/flux/:fluxId/owner-settings	✅ Yes	Get the OWNER's settings for a specific flux (for viewers).
GET	/api/flux/settings/screenshot-block	✅ Yes	Check if current user has screenshot-alert enabled.

▸ GET /api/flux/settings — Full Response

JSON Response

```
{
  "success": true,
  "data": {
    // Visibility
    "visibility":         "followers",  // default for new stories
    "audience":           "followers",  // alias
    "hideFrom":           [],             // list of hidden user IDs

    // Interactions
    "allowReplies":       "everyone",  // 'everyone' | 'mutual' | 'off'
    "allowReactions":     true,
    "allowMessageReplies":true,

    // Sharing
    "allowShareToStory":  true,
    "allowShareAsMessage":true,
    "allowExternalShare": false,

    // Save
    "saveToDevice":       false,
    "saveToArchive":      true,
    "autosaveDrafts":     true,

    // Advanced
    "duration":           24,   // 24 | 48 | 72 hours
    "showAnalytics":      true,
    "restrictScreenshots":false
  }
}
```

▸ PATCH /api/flux/settings — Update Examples

TypeScript

```
// Disable replies for all stories
await mediaApi.patch('/flux/settings', { allowReplies: 'off' });

// Set default visibility to followers-only + 48h duration
await mediaApi.patch('/flux/settings', {
  visibility: 'followers',
  duration: 48,
});

// Enable screenshot alerts
await mediaApi.patch('/flux/settings', { restrictScreenshots: true });

// Disable reactions globally
await mediaApi.patch('/flux/settings', { allowReactions: false });
```

▸ GET /api/flux/:fluxId/owner-settings — Response (used by viewers)

JSON Response

```
{
  "success": true,
  "data": {
    "allowReplies":        "everyone",
    "allowReactions":      true,
    "allowMessageReplies": true,
    "allowShareToStory":   true,
    "allowShareAsMessage": true,
    "allowExternalShare":  false,
    "saveToDevice":        false,
    "screenshotAlert":     false
  }
}
```

> Always fetch owner-settings before rendering action buttons in the flux viewer. Hide Reply, Share, and React buttons if owner has disabled them.

13 Flux Visibility & Hide-From Controls

The owner can change the visibility of individual fluxes and hide specific users from seeing any of their stories globally. Per-flux hide-from is also supported.

Method	Endpoint	Auth	Description
PATCH	/api/flux/:fluxId/visibility	✅ Yes	Update visibility of a single flux.
GET	/api/flux/:fluxId/visibility	✅ Yes	Read current visibility of a flux (owner only).
PATCH	/api/flux/:fluxId/hide-from	✅ Yes	Hide a user from seeing this specific flux.
PATCH	/api/flux/:fluxId/unhide-from	✅ Yes	Unhide a user from this specific flux.
GET	/api/flux/settings/hide-from	✅ Yes	Get global hide-from list (enriched with user info).
PATCH	/api/flux/settings/hide-from	✅ Yes	Bulk-replace the global hide-from list.
POST	/api/flux/settings/hide-from/add	✅ Yes	Add a single user to the global hide-from list.
POST	/api/flux/settings/hide-from/remove	✅ Yes	Remove a user from the global hide-from list.

▸ PATCH /api/flux/:fluxId/visibility — Body

JSON Body

```
{
  "visibility": "close_friends"
  // Options: 'public' | 'followers' | 'close_friends' | 'only_me'
}
```

▸ PATCH /api/flux/:fluxId/hide-from — Body

JSON Bodies

```
{ "hideUserId": "user_abc" }

// PATCH /:fluxId/unhide-from
{ "unhideUserId": "user_abc" }

// POST /settings/hide-from/add
{ "hideUserId": "user_abc" }

// POST /settings/hide-from/remove
{ "unhideUserId": "user_abc" }
```

14 Flux Analytics & Screenshot Reports

Analytics are only available to the flux owner and only if showAnalytics=true in their settings. Screenshot detection is optional — clients send a report event when a screenshot is taken.

Method	Endpoint	Auth	Description
GET	/api/flux/:fluxId/analytics	✅ Yes	Full analytics for a flux (owner only).
POST	/api/flux/:fluxId/screenshot	✅ Yes	Report a screenshot event for a flux.

▸ GET /api/flux/:fluxId/analytics — Response

JSON Response

```
{
  "success": true,
  "analyticsEnabled": true,
  "viewCount": 120,
  "likeCount": 34,
  "reactionBreakdown": { "😍": 20, "🔥": 10, "♡": 4 },
  "commentCount": 8,
  "mentionCount": 2,
  "screenshots": {
    "count": 5,
    "uniqueUsers": 5,
    "recentUsers": [
      { "id": "user_a", "username": "jane", "name": "Jane",
        "profile_picture": "https://..." }
    ]
  }
}
```

▸ POST /api/flux/:fluxId/screenshot — Body

TypeScript

```
{
  "platform": "ios"     // 'ios' | 'android' | 'web'
}

// React Native — detect screenshot and report it
import { addScreenshotListener } from 'react-native-screenshot-prevent';

useEffect(() => {
  const sub = addScreenshotListener(async () => {
    await mediaApi.post(`/flux/${currentFluxId}/screenshot`, {
      platform: Platform.OS
    });
  });
  return () => sub.remove();
}, [currentFluxId]);
```

> Screenshot reports are deduplicated with a 48-hour Redis gate — one notification per viewer per flux per 48h, guaranteed. Never send duplicate alerts to the owner.

15 Archive, Persistent Stories & Delete

Method	Endpoint	Auth	Description
POST	/api/flux/:fluxId/archive	✅ Yes	Toggle archive on/off for a flux (owner only).
PATCH	/api/flux/:fluxId/persistent	✅ Yes	Toggle persistent (pinned / no-expiry) story. Max 5 per user.
DELETE	/api/flux/:fluxId	✅ Yes	Soft-delete a flux. Removes media from Cloudinary.
PATCH	/api/flux/:fluxId/comments/toggle	✅ Yes	Toggle comments on/off for this specific flux.

▸ POST /api/flux/:fluxId/archive — Response

JSON Response

```
{
  "success": true,
  "isArchived": true,
  "message": "Flux archived"
}
```

▸ PATCH /api/flux/:fluxId/persistent — Response

JSON Response

```
{
  "success": true,
  "isPersistent": true,
  "expiresAt": "2124-01-01T00:00:00.000Z",
  "message": "Story is now persistent (no expiry)"
}
```

> Persistent stories appear in archive.active[] and do not expire. Limit: 5 per user. Toggling off restores the original 24h expiry from createdAt (archives immediately if already past).

16 Close Friends

Close friends are a curated list per user. Only users in this list can see 'close_friends' visibility fluxes. You must follow a user before adding them to close friends.

Method	Endpoint	Auth	Description
GET	/api/flux/close-friends	✅ Yes	Get current user's close-friends list.
GET	/api/flux/close-friends/suggestions	✅ Yes	Get follower suggestions to add as close friends.
POST	/api/flux/close-friends/add	✅ Yes	Add a single user to close friends (must be following them).
POST	/api/flux/close-friends/remove	✅ Yes	Remove a single user from close friends.
POST	/api/flux/close-friends/save	✅ Yes	Bulk-replace entire close-friends list atomically.

▸ POST /api/flux/close-friends/add — Body

JSON Body

```
{ "friendId": "user_abc" }
```

▸ POST /api/flux/close-friends/save — Body

JSON Body

```
// Replaces the entire list atomically
{ "friendIds": ["user_abc", "user_def", "user_ghi"] }
```

▸ GET /api/flux/close-friends — Response

JSON Response

```
{
  "success": true,
  "data": [
    {
      "id": "user_abc",
      "username": "janedoe",
      "name": "Jane Doe",
      "profile_picture": "https://cdn.../avatar.jpg"
    }
  ]
}
```

17 Stickers API

Giphy-powered sticker search and trending. Use these to populate the sticker picker in the story creation UI.

Method	Endpoint	Auth	Description
GET	/api/flux/stickers/trending	✅ Yes	Get trending GIF stickers (default: 20 results).
GET	/api/flux/stickers/search?q=	✅ Yes	Search stickers by keyword.

▸ GET /api/flux/stickers/trending — Response

JSON Response

```
{
  "success": true,
  "stickers": [
    {
      "id": "giphy_abc",
      "type": "gif",
      "url": "https://media.giphy.com/.../200.gif",
      "width": 200,
      "height": 200,
      "title": "Dancing Cat"
    }
  ]
}
```

▸ Query params for search

Field	Type	Required	Description
q	string	Yes	Search keyword (min 1 character).
limit	number	No	Max results (default: 20).

18 Music API (/api/music)

The music service is powered by iTunes Search API. Tracks include a 30-second preview URL for playback in the story creation picker. Trending results are cached for 30 minutes.

Method	Endpoint	Auth	Description
GET	/api/music/search?q=	✅ Yes	Search tracks by keyword. Returns up to 50 results.
GET	/api/music/trending	✅ Yes	Get curated trending tracks (top 20).
GET	/api/music/track/:trackId	✅ Yes	Get single track details by iTunes track ID.

▸ GET /api/music/search — Query Params

Field	Type	Required	Description
q	string	Yes	Search term (e.g. 'Shape of You', 'Coldplay').
limit	number	No	Max results, capped at 50. Default: 10.

▸ Response — Track Object

JSON Response

```
{
  "success": true,
  "data": [
    {
      "id": "1234567890",
      "title": "Shape of You",
      "artist": "Ed Sheeran",
      "album": "÷ (Divide)",
      "albumArt": "https://is1-ssl.mzstatic.com/image/thumb/.../100x100bb.jpg",
      "previewUrl": "https://audio-ssl.itunes.apple.com/.../preview.m4a",
      "durationMs": 233713,
      "duration": "3:53",
      "popularity": 100
    }
  ]
}
```

▸ React Native — Audio Preview

TypeScript

```
import Sound from 'react-native-sound';
// or: import TrackPlayer from 'react-native-track-player';

const playPreview = (previewUrl: string) => {
  const sound = new Sound(previewUrl, '', (err) => {
    if (!err) sound.play();
```

```
  });
};

// Store selected track fields in flux FormData
formData.append('musicId',         track.id);
formData.append('musicTitle',      track.title);
formData.append('musicArtist',     track.artist);
formData.append('musicAlbumArt',   track.albumArt ?? '');
formData.append('musicPreviewUrl', track.previewUrl ?? '');
formData.append('musicStartAt',    String(startOffsetSeconds));
```

19 Location API (/api/location)

Google Places-powered location search and details. Results are Redis-cached (search: 10 min, details: 24 h). Use in the story creation location sticker picker.

Method	Endpoint	Auth	Description
GET	/api/location/search?q=	✅ Yes	Search places by text query. Returns Place predictions.
GET	/api/location/details/:placeId	✅ Yes	Get full place details by Google Place ID.

▸ GET /api/location/search — Query Params

Field	Type	Required	Description
q	string	Yes	Search query (min 2 characters). E.g. 'Coffee shop Mumbai'.
session	string	No	Google session token UUID (for billing optimization).

▸ GET /api/location/search — Response

JSON Response

```
{
  "success": true,
  "data": [
    {
      "placeId": "ChIJN1t_tDeuEmsRUsoyG83frY4",
      "description": "Sydney Opera House, Sydney NSW, Australia",
      "mainText": "Sydney Opera House",
      "secondaryText": "Sydney NSW, Australia"
    }
  ]
}
```

▸ GET /api/location/details/:placeId — Response

JSON Response

```
{
  "success": true,
  "data": {
    "placeId": "ChIJN1t_tDeuEmsRUsoyG83frY4",
    "name": "Sydney Opera House",
    "address": "Bennelong Point, Sydney NSW 2000, Australia",
    "lat": -33.8567844,
    "lng": 151.2152967,
    "category": "tourist_attraction"
  }
}
```

▸ Attaching location to a flux

TypeScript

```
// After user picks a place from the search results:
const details = await mediaApi.get(`/location/details/${place.placeId}`);
const loc = details.data.data;

formData.append('locationLabel',    loc.name);
formData.append('locationPlaceId',  loc.placeId);
formData.append('locationLat',      String(loc.lat));
formData.append('locationLng',      String(loc.lng));
formData.append('locationCategory', loc.category ?? '');
formData.append('locationStickerX', '50');  // % position
formData.append('locationStickerY', '75');
formData.append('locationStickerTheme', '0'); // 0=dark, 1=light
```

20 Full Route Reference

All routes are relative to http://localhost:5010/api. Every route requires a Bearer token unless noted. Routes are ordered as they appear in the router (static before parameterized).

▸ Flux Routes — Static (must come before /:fluxId)

Method	Endpoint	Auth	Description
POST	/flux/create	✅ Yes	Create flux (multipart/form-data)
GET	/flux/feed	✅ Yes	Home screen story feed
GET	/flux/mine	✅ Yes	Own active stories
GET	/flux/all-mine	✅ Yes	All own stories incl. expired
GET	/flux/archive	✅ Yes	Archived + persistent stories
POST	/flux/invalidate-feed	✅ Yes	Bust feed cache after follow
GET	/flux/stickers/trending	✅ Yes	Trending Giphy stickers
GET	/flux/stickers/search	✅ Yes	Search Giphy stickers
GET	/flux/close-friends	✅ Yes	My close-friends list
GET	/flux/close-friends/suggestions	✅ Yes	Follower suggestions for CF
POST	/flux/close-friends/add	✅ Yes	Add to close friends
POST	/flux/close-friends/remove	✅ Yes	Remove from close friends
POST	/flux/close-friends/save	✅ Yes	Bulk save close-friends list
GET	/flux/settings	✅ Yes	Get my story settings
PATCH	/flux/settings	✅ Yes	Update story settings
GET	/flux/settings/hide-from	✅ Yes	Global hide-from list
PATCH	/flux/settings/hide-from	✅ Yes	Bulk replace hide-from list
POST	/flux/settings/hide-from/add	✅ Yes	Add to hide-from list
POST	/flux/settings/hide-from/remove	✅ Yes	Remove from hide-from list
GET	/flux/settings/screenshot-block	✅ Yes	Check screenshot-alert setting
GET	/flux/user/:userId	✅ Yes	Active stories for another user

▸ Flux Routes — Parameterized (/:fluxId)

Method	Endpoint	Auth	Description
GET	/flux/:fluxId	✅ Yes	Single flux by ID
DELETE	/flux/:fluxId	✅ Yes	Delete flux (owner only)
GET	/flux/:fluxId/analytics	✅ Yes	Flux analytics (owner only)
POST	/flux/:fluxId/screenshot	✅ Yes	Report screenshot event
GET	/flux/:fluxId/owner-settings	✅ Yes	Owner's interaction settings
PATCH	/flux/:fluxId/persistent	✅ Yes	Toggle persistent/pinned
POST	/flux/:fluxId/view	✅ Yes	Legacy view record
POST	/flux/:fluxId/record-view	✅ Yes	Deduped view record
GET	/flux/:fluxId/view-list	✅ Yes	Raw viewer IDs (owner)
GET	/flux/:fluxId/viewers	✅ Yes	Enriched viewers + reactions
POST	/flux/:fluxId/react	✅ Yes	Emoji reaction
POST	/flux/:fluxId/like	✅ Yes	Toggle like
GET	/flux/:fluxId/likes	✅ Yes	Get likes
POST	/flux/:fluxId/comments	✅ Yes	Add comment
GET	/flux/:fluxId/comments	✅ Yes	Get comments
POST	/flux/:fluxId/comments/:id/like	✅ Yes	Toggle comment like
PATCH	/flux/:fluxId/comments/toggle	✅ Yes	Enable/disable comments
POST	/flux/:fluxId/archive	✅ Yes	Toggle archive
POST	/flux/:fluxId/reply	✅ Yes	Reply as DM chat message
POST	/flux/:fluxId/share	✅ Yes	Share to user DMs
POST	/flux/:fluxId/mention	✅ Yes	Mention users (owner only)
GET	/flux/:fluxId/mentions	✅ Yes	Get mentioned users
DELETE	/flux/:fluxId/mention/remove	✅ Yes	Self-remove from mentions
POST	/flux/:fluxId/remention	✅ Yes	Re-mention (reshare)
GET	/flux/:fluxId/rementions	✅ Yes	Get re-mentions
GET	/flux/:fluxId/permissions	✅ Yes	isOwner + isMentioned
PATCH	/flux/:fluxId/visibility	✅ Yes	Update per-flux visibility
GET	/flux/:fluxId/visibility	✅ Yes	Read per-flux visibility
PATCH	/flux/:fluxId/hide-from	✅ Yes	Hide user from this flux
PATCH	/flux/:fluxId/unhide-from	✅ Yes	Unhide user from this flux

▸ Music & Location Routes

Method	Endpoint	Auth	Description
GET	/music/search?q=	✅ Yes	Search tracks (iTunes)
GET	/music/trending	✅ Yes	Trending tracks
GET	/music/track/:trackId	✅ Yes	Single track details
GET	/location/search?q=	✅ Yes	Google Places autocomplete
GET	/location/details/:placeId	✅ Yes	Google Place full details

21 React Native Integration Examples

▸ Complete mediaService.ts for React Native

TypeScript

```
// src/services/mediaService.ts
import AsyncStorage from '@react-native-async-storage/async-storage';
import axios from 'axios';
import Config from 'react-native-config';

const BASE = Config.MEDIA_API_URL || 'http://10.0.2.2:5010/api';

const mediaApi = axios.create({ baseURL: BASE, timeout: 30000 });

mediaApi.interceptors.request.use(async (cfg) => {
  const token = await AsyncStorage.getItem('token');
  if (token) cfg.headers.Authorization = `Bearer ${token}`;
  return cfg;
});

// ── Flux ──────────────────────────────────────────────────────────
export const getMyFluxes      = () =>
  mediaApi.get('/flux/mine').then(r => r.data.data);

export const getFluxFeed      = () =>
  mediaApi.get('/flux/feed').then(r => r.data.data);

export const recordFluxView   = (id: string) =>
  mediaApi.post(`/flux/${id}/record-view`).then(r => r.data);

export const getFluxViewers   = (id: string) =>
  mediaApi.get(`/flux/${id}/viewers`).then(r => r.data);

export const reactToFlux      = (id: string, emoji: string) =>
  mediaApi.post(`/flux/${id}/react`, { emoji });

export const toggleFluxLike   = (id: string) =>
  mediaApi.post(`/flux/${id}/like`).then(r => r.data);

export const replyToFlux      = (id: string, text: string) =>
  mediaApi.post(`/flux/${id}/reply`, { text }).then(r => r.data);

export const shareFlux        = (id: string, receiverIds: string[]) =>
  mediaApi.post(`/flux/${id}/share`, { receiverIds }).then(r => r.data);

export const deleteFlux       = (id: string) =>
  mediaApi.delete(`/flux/${id}`);

export const getFluxSettings  = () =>
  mediaApi.get('/flux/settings').then(r => r.data.data);

export const updateFluxSettings = (patch: Record<string, any>) =>
  mediaApi.patch('/flux/settings', patch).then(r => r.data.data);
```

```
export const getOwnerSettings = (id: string) =>
  mediaApi.get(`/flux/${id}/owner-settings`).then(r => r.data.data);

// ── Music ──────────────────────────────────────────────────────────
export const searchMusic      = (q: string, limit = 10) =>
  mediaApi.get('/music/search', { params: { q, limit } }).then(r => r.data.data);

export const getTrendingMusic = () =>
  mediaApi.get('/music/trending').then(r => r.data.data);

// ── Location ───────────────────────────────────────────────────────
export const searchLocation   = (q: string, session?: string) =>
  mediaApi.get('/location/search', { params: { q, session } }).then(r =>
r.data.data);

export const getPlaceDetails  = (placeId: string) =>
  mediaApi.get(`/location/details/${placeId}`).then(r => r.data.data);
```

▸ Flux Creation Screen — Full Flow

TypeScript

```
// src/screens/CreateFluxScreen.tsx  (simplified)
const handlePublish = async () => {
  setUploading(true);
  try {
    const fd = new FormData();

    // 1. Attach media
    if (selectedAsset) {
      fd.append('media', {
        uri: selectedAsset.uri,
        name: selectedAsset.fileName || 'story.jpg',
        type: selectedAsset.type || 'image/jpeg',
      } as any);
    }

    // 2. Core metadata
    fd.append('visibility', visibility);      // 'public' | 'followers' | ...
    if (caption) fd.append('caption', caption);

    // 3. Music (if user picked a track)
    if (selectedTrack) {
      fd.append('musicId',         selectedTrack.id);
      fd.append('musicTitle',      selectedTrack.title);
      fd.append('musicArtist',     selectedTrack.artist);
      fd.append('musicPreviewUrl', selectedTrack.previewUrl || '');
      fd.append('musicAlbumArt',   selectedTrack.albumArt  || '');
      fd.append('musicStartAt',    String(trimStart));
    }

    // 4. Location sticker (if user picked a place)
    if (selectedPlace) {
      fd.append('locationLabel',   selectedPlace.name);
      fd.append('locationPlaceId', selectedPlace.placeId);
      fd.append('locationLat',     String(selectedPlace.lat));
      fd.append('locationLng',     String(selectedPlace.lng));
```

```
      fd.append('locationStickerX','50');
      fd.append('locationStickerY','75');
    }

    // 5. Sticker overlays
    if (stickers.length > 0)
      fd.append('stickers', JSON.stringify(stickers));

    // 6. Text layers (text-story mode)
    if (textLayers.length > 0) {
      fd.append('textLayers', JSON.stringify(textLayers));
      fd.append('textBg',     textBackground);
    }

    const res = await mediaApi.post('/flux/create', fd, {
      headers: { 'Content-Type': 'multipart/form-data' },
      onUploadProgress: e =>
        setProgress(Math.round((e.loaded * 100) / (e.total || 1))),
    });

    console.log('Flux created:', res.data.data._id);
    navigation.goBack();
  } catch (err) {
    Alert.alert('Upload failed', String(err));
  } finally {
    setUploading(false);
  }
};
```

▸ Flux Viewer — Home Screen FlatList

TypeScript

```
// src/screens/HomeScreen.tsx — story tray
const [feed, setFeed] = useState([]);

useEffect(() => {
  getFluxFeed().then(setFeed);
}, []);

const renderStoryBubble = ({ item }) => {
  const hasUnseen = item.fluxes.some(f => !f.viewedByMe);
  return (
    <TouchableOpacity onPress={() =>
      navigation.navigate('FluxViewer', { ownerId: item._id, fluxes: item.fluxes })
    }>
      {hasUnseen ? (
        <LinearGradient colors={['#8860D9','#B3B8E2']}
          style={{ padding: 2, borderRadius: 999 }}>
          <Image source={{ uri: item.user?.profile_picture }} style={styles.bubble}
/>
        </LinearGradient>
      ) : (
        <Image source={{ uri: item.user?.profile_picture }} style={styles.bubble}
/>
      )}
      <Text>{item.isSelf ? 'Your Story' : item.user?.name}</Text>
```

```
    </TouchableOpacity>
  );
};

return (
  <FlatList
    horizontal data={feed}
    keyExtractor={i => i._id}
    renderItem={renderStoryBubble}
  />
);
```